sglang-omni 阶段通信

中文译文与科普图 · Communication

在 sglang-omni 的阶段(Stage)间通信中，ZMQ 负责协调信息和序列化后的控制元数据(Control Metadata)，`sglang_omni.comm` 则定义数据搬运规则。阶段代码根据阶段名称路由，由通信路由器(Comm Router)选择具体路径。

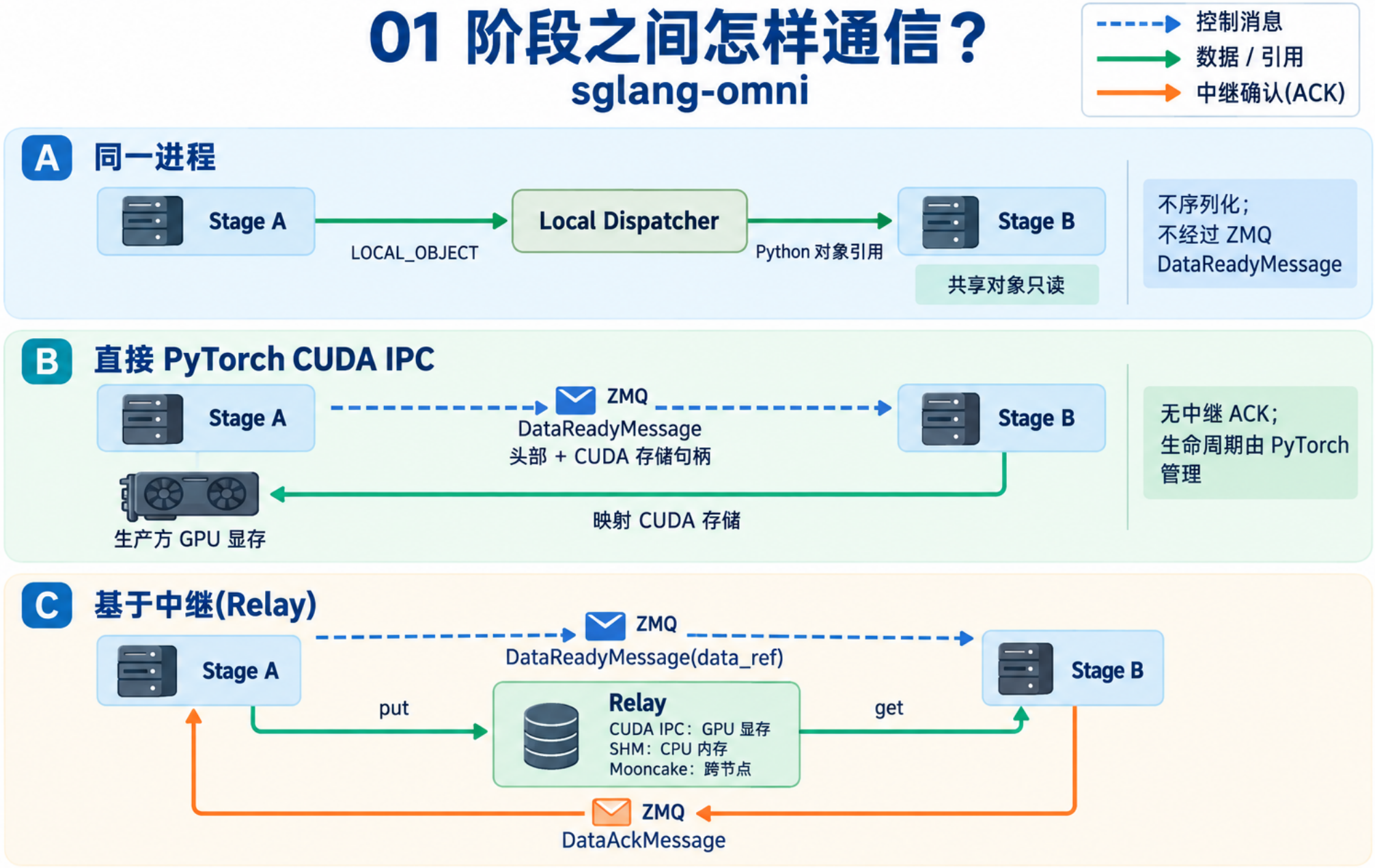
可选路径包括：同进程对象传递；同一部署位置(Placement)、进程内 CUDA 设备序号兼容时的直接 PyTorch CUDA IPC；同节点其他 GPU 链路的池化 CUDA IPC；本地 CPU 数据搬运使用共享内存(SHM)；已配置的跨节点搬运使用 Mooncake。

主要实现入口

文件	职责
sglang_omni/comm/data_ref.py	定义由 DataReadyMessage.data_ref 携带的、具有明确类型的中继数据引用 DataRef
sglang_omni/comm/router.py	判断位置关系(Locality)，选择传输方式(Transport)
sglang_omni/comm/engine.py	面向阶段代码的统一通信接口
sglang_omni/comm/stage_io.py	打包和解包载荷(Payload)及流式张量(Stream Tensor)
sglang_omni/pipeline/control_plane.py	ZMQ 套接字、msgpack 序列化，以及阶段与协调器(Coordinator)的消息路由
sglang_omni/pipeline/local_dispatch.py	同进程共同部署阶段之间的 Python 对象分发
sglang_omni/relay/base.py	中继(Relay)后端接口和后端注册表
sglang_omni/relay/cuda_ipc.py	发送方拥有的 CUDA 显存池、槽位分配、数据复制和完成处理
sglang_omni/relay/{shm,nccl,nixl,mooncake}.py	具体中继后端
sglang_omni/proto/messages.py	控制平面(Control Plane)消息类型

图 1 展示同进程对象传递、直接 CUDA IPC 和中继传输三条主要路径。

01 阶段之间怎样通信？
sglang-omni



小型 CPU 流式数据可内联控制消息，见图 03

图 1 通信总览：控制消息、数据与对象引用的传递路径。

传输模型(Transfer Model)

路径 / 传输方式	携带内容
协调 / ZMQ PUSH/PULL	SubmitMessage、DataReadyMessage、CompleteMessage、StreamMessage、ShutdownMessage，以及性能分析器(Profiler)控制消息
广播协调 / ZMQ PUB/SUB	AbortMessage
同进程 / LOCAL_OBJECT	同一操作系统进程内，通过 Python 引用传递完整 StagePayload 对象和流式数据块(Stream Chunk)
同部署位置的直接 GPU 搬运 / PyTorch CUDA IPC	CUDA 存储句柄，以及普通载荷或流式传输的控制元数据
同节点的池化 GPU 搬运 / CUDA IPC 中继	打包后的载荷张量缓冲区、CUDA 流式数据块，以及流式元数据张量
本地 CPU 搬运 / SHM 中继	完整载荷的张量缓冲区，以及不走本地 CUDA 数据通路的流式数据块
跨节点搬运 / Mooncake 中继	经 Mooncake 选择的传输方式搬运完整载荷的张量缓冲区和流式数据块

`DataReadyMessage.data_ref` 携带直接 PyTorch CUDA IPC 封装(Envelope)、内联 CPU 流式封装，或具有明确类型的中继 `DataRef`。直接封装包含经 pickle 序列化的载荷头部或流式元数据，以及 PyTorch CUDA 存储句柄。内联 CPU 流式封装包含序列化后的张量及不含张量的元数据。

中继 `DataRef` 包含对象 ID、数据种类、传输方式、布局、后端缓冲区引用、张量布局，以及可选的流式元数据。`RelayOperation.metadata` 中由后端管理的细节存放在 `DataRef.buffer.info` 下。

普通载荷流程(Normal Payload Flow)

协调器通过 `SubmitMessage` 将第一个 `StagePayload` 直接提交给入口阶段。此后，阶段间载荷根据链路和载荷情况，使用 LOCAL_OBJECT、直接 PyTorch CUDA IPC 或中继传输。

直接 CUDA IPC 路径中，`Stage` 提取 CUDA 张量叶子节点，将剩余 `StagePayload` 用 pickle 序列化为普通控制元数据，并通过 PyTorch 的 CUDA 多进程序列化器(Reducer)序列化每个 CUDA 张量。随后发送一条包含这些存储句柄的 `DataReadyMessage`。接收方映射生产方分配的 GPU 显存并恢复载荷，无须中继缓冲区。此路径没有中继 `DataAckMessage`，生命周期由 PyTorch 的 CUDA IPC 所有权机制管理。

普通直接载荷的头部属于 ZMQ 控制元数据。其大小与池化 CUDA 中继的槽位大小无关；较大的头部不会被拆分成中继槽位或应用层消息。

普通载荷流程：中继与同进程路径

对于基于中继的载荷，流程如下：

① 发送方向 `CommRouter` 查询链路传输方式，并调用 `CommEngine.send_payload(...)`。

② 发送工作线程调用 `stage_io.write_payload()`：递归提取 `payload.data` 中的张量，用占位符替换；将不含张量的 `StagePayload` 用 pickle 序列化，并将张量拼接成一个 `uint8` 缓冲区。

③ 发送方对该缓冲区调用 `relay.put_async()`，再发送 `DataReadyMessage(data_ref=...)`。其中 `DataRef` 包含：`buffer.info`，即来自 `RelayOperation.metadata` 的后端专用元数据；`header`，即不含张量、经 base64 编码的 `StagePayload`；`tensors`，即每个张量的路径、形状(Shape)、数据类型(Dtype)、偏移量和字节数。

④ 接收方在 `Stage._on_data_ready()` 中处理消息，调用 `CommEngine.read_payload()`，等待 `relay.get_async()` 完成，恢复张量，并将载荷交给阶段输入处理器。

⑤ 接收方发送一条 `DataAckMessage`，发送方随后释放为该逻辑封装保留的操作。

⑥ 如果输入汇聚(Fan-in)完成，阶段将一条 `IncomingMessage` 放入 `scheduler.inbox`。

中继载荷格式刻意保持与后端无关。后端只需要搬运一个扁平张量缓冲区，并返回可供另一后端实例调用 `get_async()` 的元数据。

LOCAL_OBJECT 绕过中继和 ZMQ `DataReadyMessage`：发送方调用进程内分发器，由其调用目标阶段的 `receive_local_payload()`，直接传入投影函数生成的 `StagePayload` 对象。这是 Python 引用传递，不经过序列化。

接收方必须将载荷、嵌套数据容器、张量、流式数据块和元数据视为只读；对象在接收方调度器队列中的整个存续期间也必须保持有效。发送方和投影函数在分发后不得修改或回收这些对象。

请求和控制对象应保留下游需要的参数，但不应在后续阶段传递中继续持有已消费的大体积媒体数据。将原始媒体转换为流水线规范状态的阶段负责释放这些引用。

对于完整载荷，同进程、单目标路由可以使用 LOCAL_OBJECT。对于扇出(Fan-out)，只有当每个投影后的载荷都是拥有独立 `data` 容器的 `StagePayload` 时，才允许使用 LOCAL_OBJECT，确保下游阶段不共享可变载荷状态。张量叶子节点仍可有意共享，但必须只读使用。

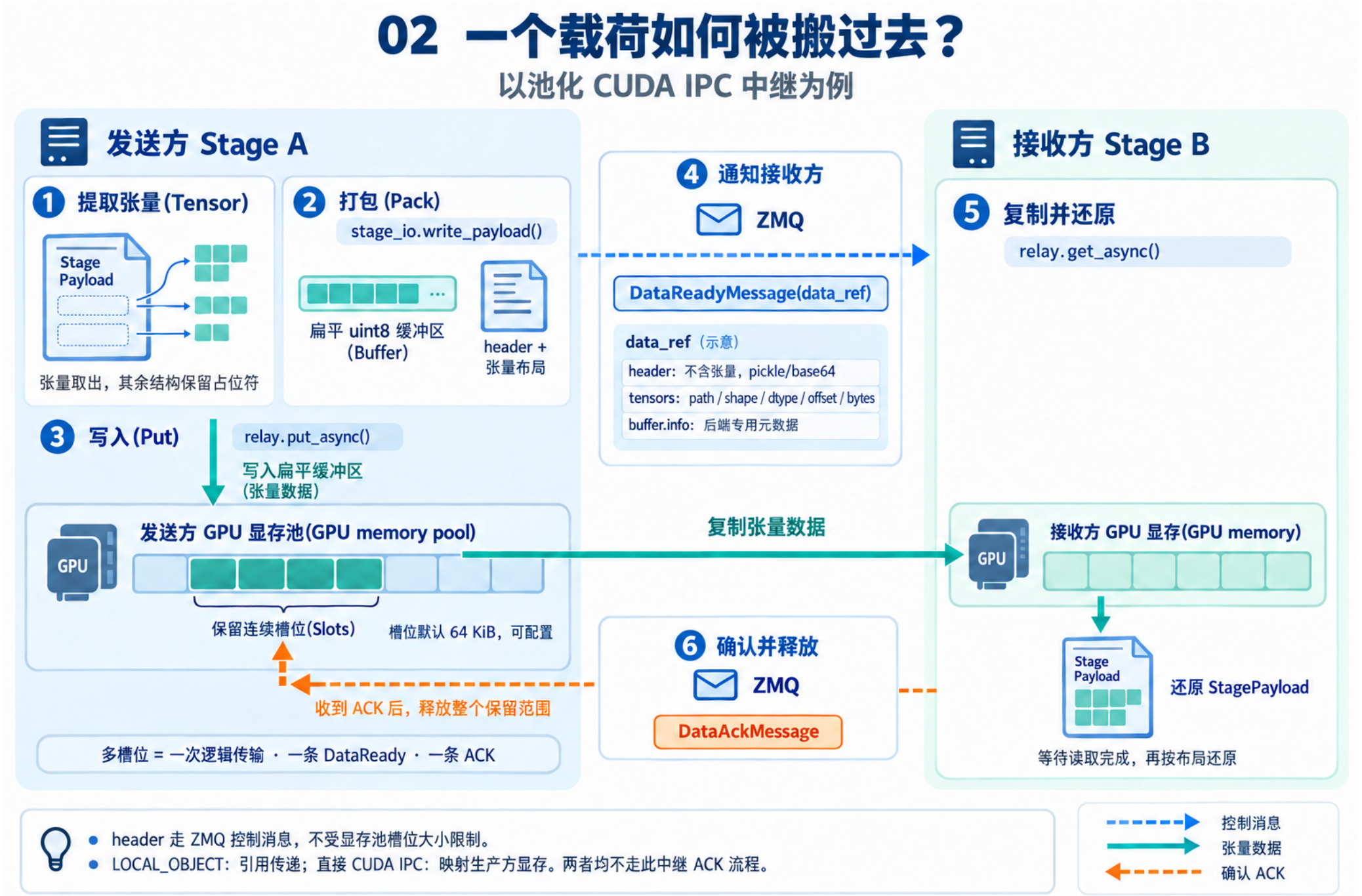


图 2 普通载荷：以池化 CUDA IPC 中继说明打包、复制、还原与 ACK 后释放。

流式传输流程(Streaming Flow)

流式传输用于生产者—消费者链路，例如 thinker 向 talker 传递隐藏状态(Hidden States)，或 talker 向 vocoder 传递编码张量(Code Tensors)。阶段层提供统一发送辅助方法 `CommEngine.send_stream_chunk()`，由路由器选择传输方式。

同节点 GPU 目标

同一部署位置且命名空间兼容(Namespace-compatible)的进程，可以通过直接 PyTorch CUDA IPC 封装发送 CUDA 数据块。

直接流式元数据可以包含 CUDA 张量和普通内联值，但不能包含 CPU 张量；直接编解码器(Codec)另有独立的 64 KiB 内联元数据准入上限。

同节点的其他 GPU 链路使用池化 CUDA IPC 中继。其流式 `DataReadyMessage` 携带一个 `DataRef`，其中包含 `transport="cuda_ipc"` 和 `chunk_id`。

同进程流式目标

阶段通过一次 `LocalStageDispatcher.send_stream_chunks()` 调用，按顺序向同一目标发送连续数据块。

接收方仍逐块检查，并为每个数据块入队一条 `stream_chunk` 消息；接收方通过引用获得原始 Python 对象和元数据。同样适用载荷 LOCAL_OBJECT 的只读和生命周期要求。

非进程内的流式目标

如果 CPU 张量及其元数据序列化后的总大小不超过 16 KiB，且元数据不含张量，可以直接放入 `DataReadyMessage`。内联封装拥有自身的字节数据，因此无须中继 ACK；超过大小上限的数据块继续通过所选中继传输。

数据块通过 `write_tensor()` 写入；元数据中值为张量的部分被提取，并分别写成独立的 `DataRef`。先发送控制消息，再等待尚未完成的写入操作(Put Operation)。

接收方在 `Stage._on_stream_chunk()` 中读取数据块，并将一条 `stream_chunk` 消息放入 `scheduler.inbox`。

“先发送控制消息，再等待”的顺序对 NIXL 和其他基于额度(Credit)的后端十分重要。如果发送方在通知接收方之前就等待完成，接收方将无法启动读取，而该读取正是释放发送方额度所必需的操作。

流式传输完成和错误通过 `send_stream_signal()` 发送，属于仅含控制信息的消息。

03 流式数据为什么要先通知？

逐块传输，先发控制消息，再等待写入完成

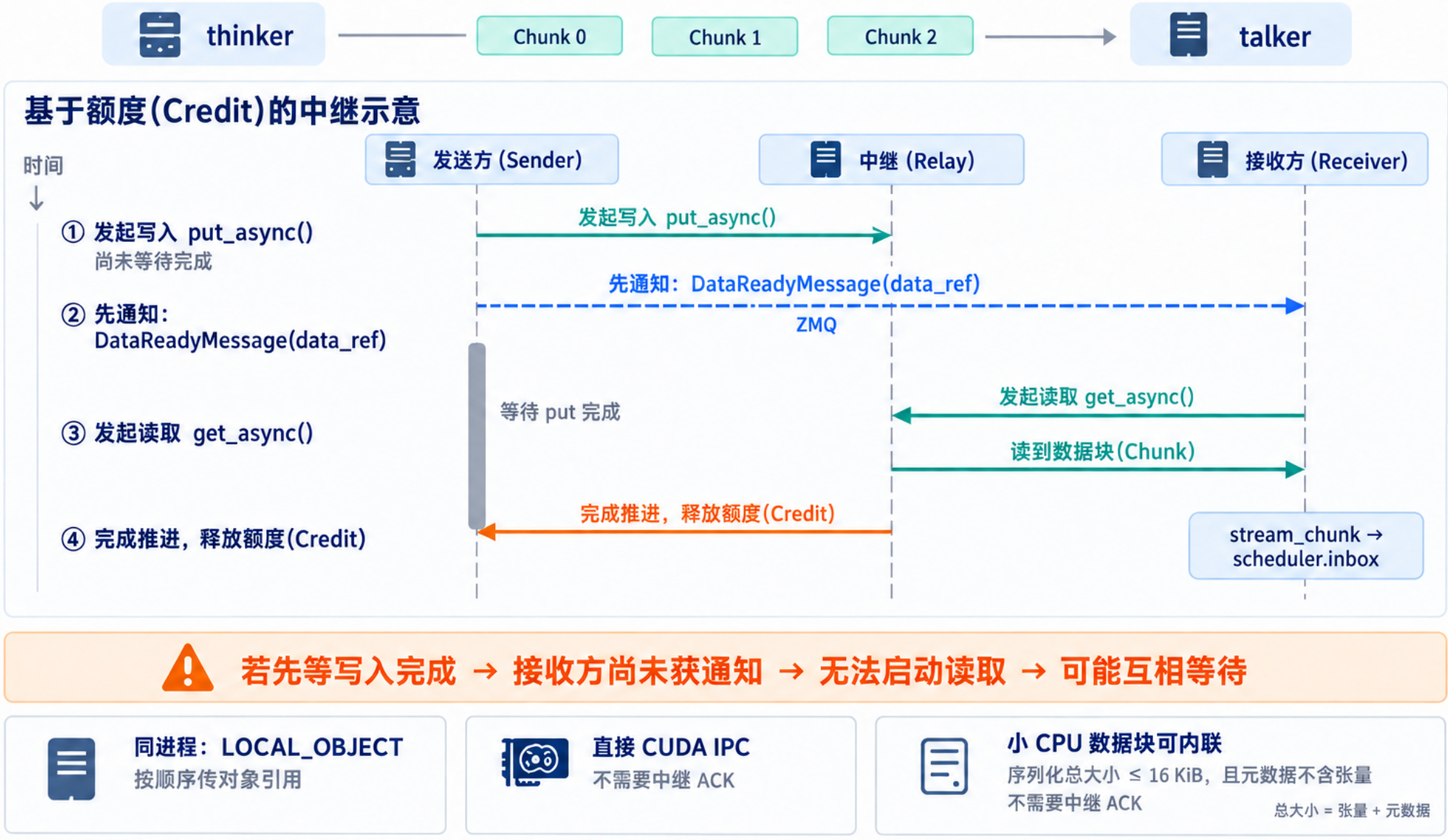


图 3 流式传输：基于额度的中继需要先通知接收方，再等待写入完成。

中继接口(Relay Interface)

所有后端均实现 `Relay`：

```
class Relay:
    async def put_async(
        self, tensor: torch.Tensor, request_id: str | None = None,
        dst_rank: int | None = None
    ) -> RelayOperation: ...

    async def get_async(
        self, metadata: Any, dest_tensor: torch.Tensor,
        request_id: str | None = None
    ) -> RelayOperation: ...

    def cleanup(self, request_id: str) -> None: ...
    def close(self) -> None: ...
```

`put_async()` 返回一个 `RelayOperation`，其 `metadata` 被放入控制消息。写入和读取操作都提供 `await wait_for_completion(timeout=...)`。阶段会保留操作对象，直到传输可以安全释放。

CUDA IPC 中继管理一个容量有限、归发送方所有的 GPU 显存池。其分配粒度默认为 64 KiB，可通过 `cuda_ipc_slot_size_kb` 配置。一个张量可以预留多个连续槽位，但这些槽位仍属于一次逻辑传输：发送方发布一条 `DataReadyMessage`，接收方从导出的显存池范围复制数据，再通过一条逻辑 `DataAckMessage` 释放整个范围。槽位表示分配器粒度，不是应用层分页。

传输方式选择(Transport Selection)

没有公开的后端选择接口。`CommRouter` 根据阶段的位置关系和部署位置推导传输方式：

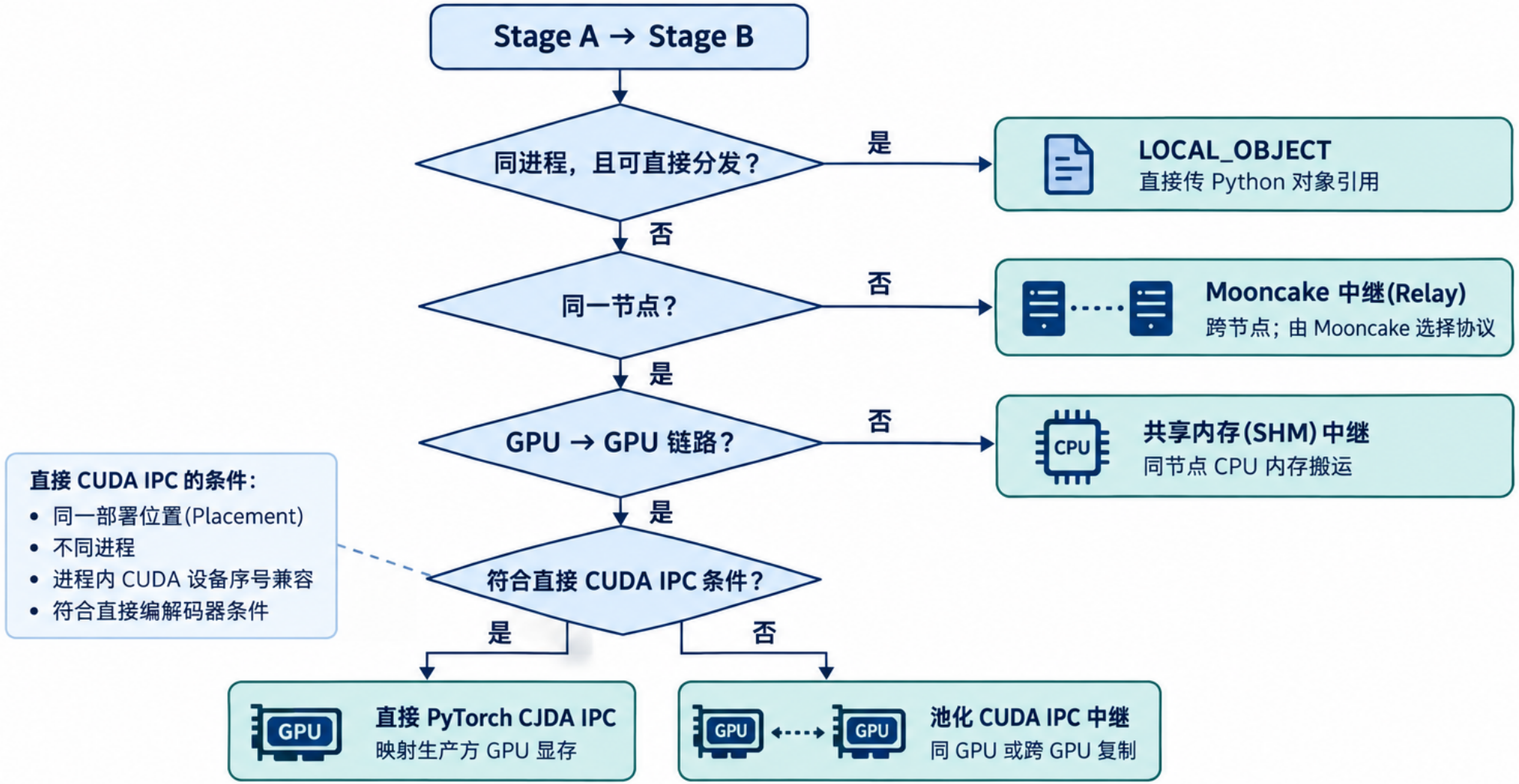
传输方式	选择规则
local_object	源阶段和目标阶段位于同一操作系统进程，且载荷符合直接本地分发条件
直接 PyTorch CUDA IPC	同一部署位置、不同进程，运行时能够确认进程内 CUDA 设备序号兼容；载荷或流式数据块还须符合直接编解码器的要求
cuda_ipc	未使用直接 PyTorch CUDA IPC 的同节点 GPU 链路；池化中继支持同 GPU 和跨 GPU 搬运
shm	同节点主机/CPU 传输，且所选链路不是 GPU 到 GPU
mooncake	被列为远程的跨节点阶段链路；由 Mooncake 选择这些传输使用的协议

`CommConfig` 可以按阶段调整槽位大小、额度和 Mooncake 连接选项，但不负责选择传输后端。每个后端只负责传输机制，不负责请求路由、输入汇聚、选择下游阶段或解释模型载荷。

04 传输路径是怎么选出来的？

CommRouter 根据位置关系与载荷条件自动选择

普通载荷示例；跨节点链路已配置 remote



LOCAL_OBJECT：扇出(Fan-out)需各自独立的 data 容器；共享张量只读。

流式特例：小型 CPU 数据块可内联控制消息，见图 03。

CommConfig 调整参数；传输路径由 CommRouter 选择。

图 4 路由决策：以普通载荷为例，跨节点链路须已配置为 remote。

资源生命周期(Resource Lifetime)

阶段层遵循以下所有权规则：

发送方写入数据，发送 `DataReadyMessage`，然后按后端要求等待写入操作完成。

接收方分配目标缓冲区，等待读取操作完成，恢复载荷，并调用 `relay.cleanup(request_id)`。

池化 CUDA IPC 的发送方保留完整槽位范围，直到接收方的一条逻辑 ACK 将该封装的所有操作标记为完成。

直接 PyTorch CUDA IPC 没有中继 ACK，依赖 PyTorch 导入存储的生命周期管理。

LOCAL_OBJECT 没有后端清理操作。发送方与接收方共享 Python 对象引用，正确性依赖于接收方使用结束前始终只读。

中止请求时，阶段中止路径调用 `relay.cleanup(request_id)`；阶段关闭时调用 `relay.close()`。

后端专用清理细节封装在该接口后面。例如，`shm` 在接收时解除共享 CPU 内存块的链接(Unlink)，NIXL 和 Mooncake 在完成后释放缓冲池额度，NCCL 在关闭时销毁进程组(Process Group)。